Geoscientific Machine Learning

Pankaj K Mishra

Table of Contents

1 Geoscientific Machine Learning

Welcome to the book **Geoscientific Machine Learning**. This book will teach you basics of Julia programming, machine learning, and how to mix geoscience domain knowledge with machine learning methods.

> ⚠ Free living book
>
> This is a living book and free for everyone! Contents will be dynamically updated at regular inervals betweeen major released versions.

> ⚠ This book is under construction
>
> Contents are not ready for readers. If you want me to include something specific while I work on this book part-time Please open an issue and I'll try to add it.

Acknowledgements

This is a living book, continuously updated with new content and examples.

2 Getting started with Julia

2.1 Setting up VS Code

1. Download and install Visual Studio Code
2. Install the **Julia** extension from the VS Code marketplace
3. Configure the Julia path in VS Code settings if needed

2.1.1 Recommended Extensions

- **Julia Language Support** - syntax highlighting, code completion
- **Quarto** - for rendering notebooks and documents

2.2 Installing Julia

Download Julia from julialang.org.

2.2.1 Windows

1. Download the Windows installer (.exe)
2. Run the installer and follow the prompts
3. Add Julia to your PATH (optional but recommended)

2.2.2 macOS

1. Download the macOS .dmg file
2. Drag Julia to Applications

2.2.3 Linux

```
# Using juliaup (recommended)
curl -fsSL https://install.julialang.org | sh
```

2.3 Basic numerics

```
1 + 2, sin(1.0)
```

```
(3, 0.8414709848078965)
```

2.3.1 Arithmetic

```
a = 10
b = 3
a + b, a - b, a * b, a / b
```

```
(13, 7, 30, 3.3333333333333335)
```

2.3.2 Arrays

```
x = [1, 2, 3, 4, 5]
sum(x), length(x)
```

```
(15, 5)
```

2.4 Functions

```
f(x) = x^2 + 2x + 1
f(3)
```

```
16
```

2.5 Control Flow

2.5.1 Conditionals

```
x = 5
if x > 0
    println("positive")
elseif x < 0
    println("negative")
else
    println("zero")
end
```

```
positive
```

2.5.2 Loops

```
for i in 1:5
    println(i^2)
end
```

```
1
4
9
16
25
```

2.6 Types and Structs

```
struct Point
    x::Float64
    y::Float64
end

p = Point(1.0, 2.0)
p.x, p.y
```

```
(1.0, 2.0)
```

3 Files & Data Manipulation

3.1 Reading and Writing Files

3.1.1 Text Files

```
# Writing to a file
open("example.txt", "w") do f
    write(f, "Hello, Julia!")
end

# Reading from a file
content = read("example.txt", String)
println(content)
```

```
Hello, Julia!
```

3.1.2 CSV Files

```
using CSV, DataFrames

# Read CSV
df = CSV.read("data.csv", DataFrame)

# Write CSV
CSV.write("output.csv", df)
```

3.2 DataFrames

DataFrames are tabular data structures similar to pandas in Python or data.frames in R.

3.2.1 Creating DataFrames

```
using DataFrames

# Create from columns
df = DataFrame(
    name = ["Alice", "Bob", "Charlie"],
    age = [25, 30, 35],
    city = ["Helsinki", "Espoo", "Tampere"]
)
```

	name	age	city
	String	Int64	String
1	Alice	25	Helsinki
2	Bob	30	Espoo
3	Charlie	35	Tampere

3.2.2 Basic Operations

```
# Select columns
df[:, :name]
```

```
3-element Vector{String}:
 "Alice"
 "Bob"
 "Charlie"
```

```
# Filter rows
filter(row -> row.age > 25, df)
```

	name	age	city
	String	Int64	String
1	Bob	30	Espoo
2	Charlie	35	Tampere

```
# Add new column
df.country = ["Finland", "Finland", "Finland"]
df
```

	name	age	city	country
	String	Int64	String	String
1	Alice	25	Helsinki	Finland
2	Bob	30	Espoo	Finland
3	Charlie	35	Tampere	Finland

3.2.3 Grouping and Aggregation

```
using Statistics

# Group by and summarize
gdf = groupby(df, :country)
combine(gdf, :age => mean => :avg_age)
```

	country	avg_age
	String	Float64
1	Finland	30.0

3.3 Working with Geoscientific Data

3.3.1 Loading Geophysical Data

```
using DelimitedFiles

# Read space-delimited data
data = readdlm("gravity_data.txt")

# Extract columns
x, y, gz = data[:, 1], data[:, 2], data[:, 3]
```

3.3.2 NetCDF Files

```
using NCDatasets

# Read NetCDF
ds = Dataset("climate_data.nc")
temp = ds["temperature"][:]
close(ds)
```

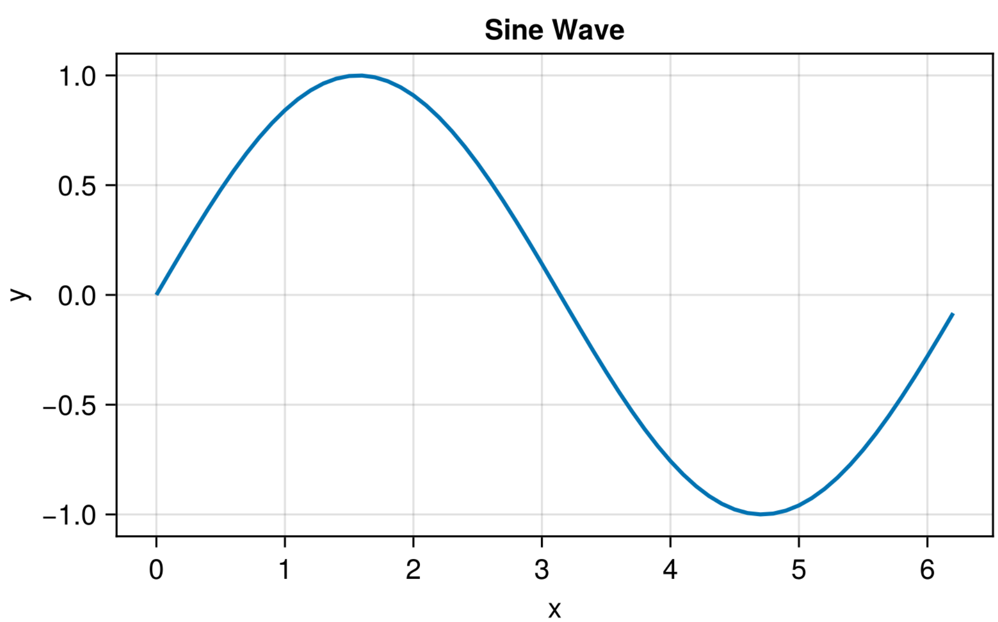
4 Plotting & Maps

4.1 Basic Plotting with CairoMakie

CairoMakie is a powerful plotting package that produces high-quality vector graphics and supports direct PDF output.

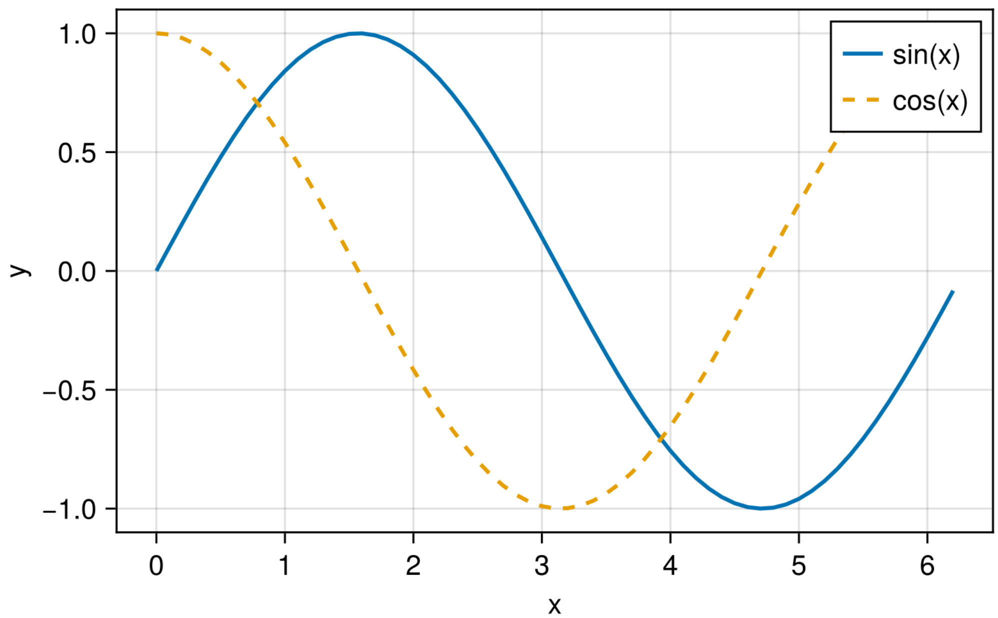
```julia
using CairoMakie
CairoMakie.activate!(type = "png")
```

```julia
x = 0:0.1:2π
y = sin.(x)
fig = lines(x, y, axis=(xlabel="x", ylabel="y", title="Sine Wave"),
            label="sin(x)", linewidth=2)
fig
```



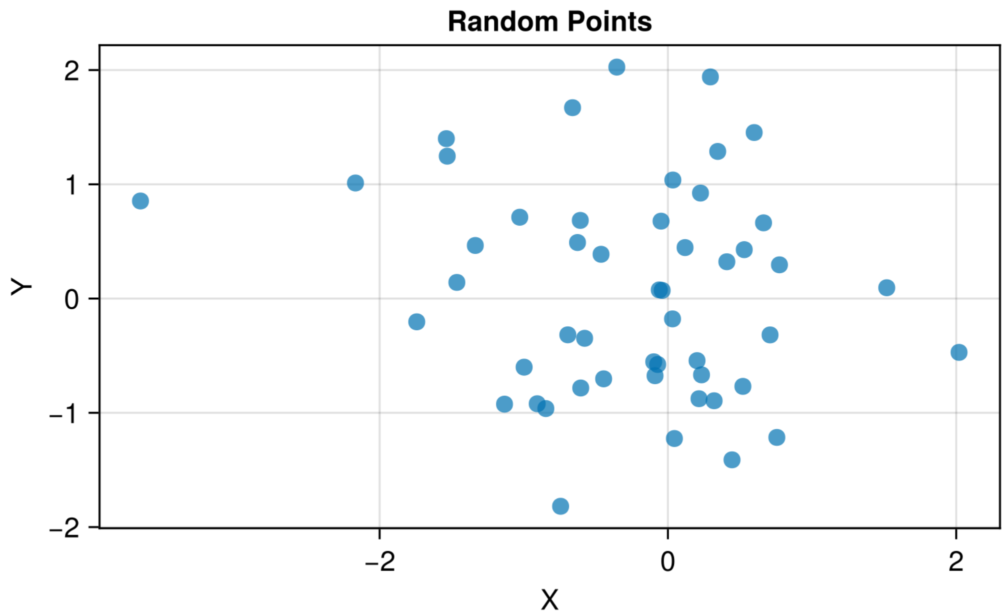
4.1.1 Multiple Series

```
fig = Figure()
ax = Axis(fig[1, 1], xlabel="x", ylabel="y")
lines!(ax, x, sin.(x), label="sin(x)", linewidth=2)
lines!(ax, x, cos.(x), label="cos(x)", linewidth=2, linestyle=:dash)
axislegend(ax)
fig
```



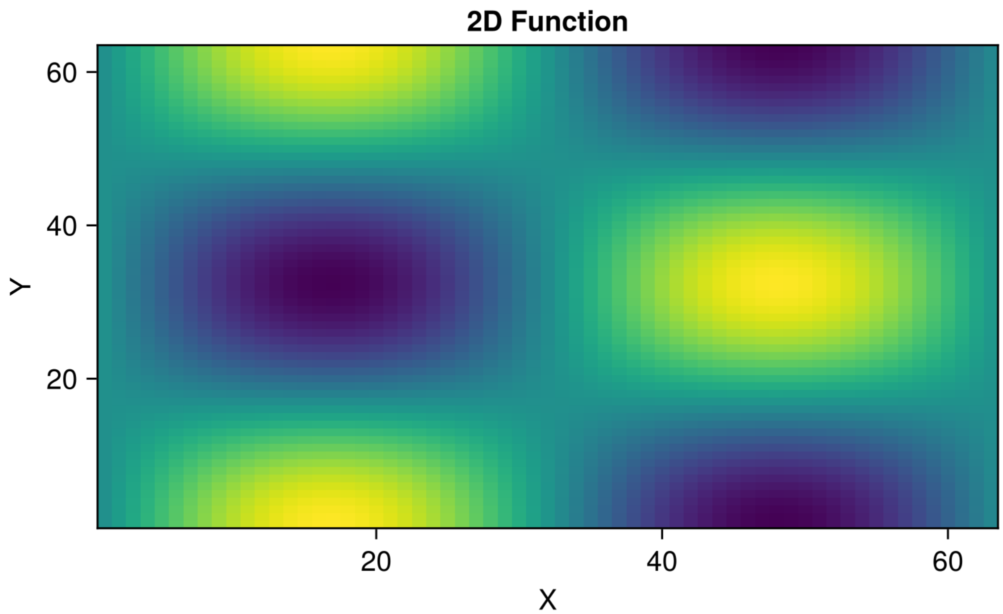
4.1.2 Scatter Plots

```
xs = randn(50)
ys = randn(50)
fig = scatter(xs, ys,
              axis=(xlabel="X", ylabel="Y", title="Random Points"),
              markersize=12, alpha=0.7)
fig
```



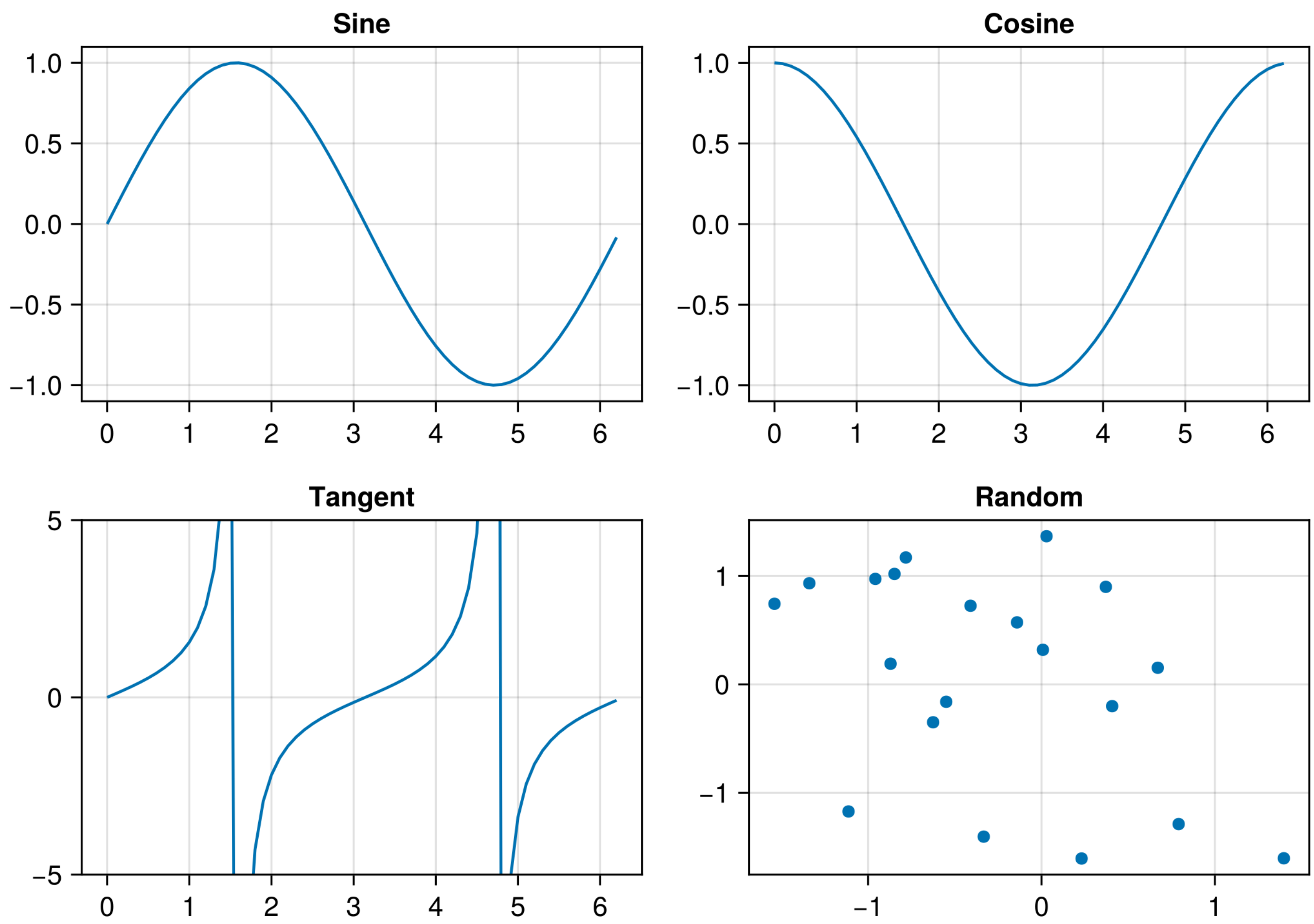
4.1.3 Heatmaps

```
z = [sin(xi) * cos(yi) for xi in 0:0.1:2π, yi in 0:0.1:2π]
fig = heatmap(z, axis=(xlabel="X", ylabel="Y", title="2D Function"),
              colormap=:viridis)
fig
```



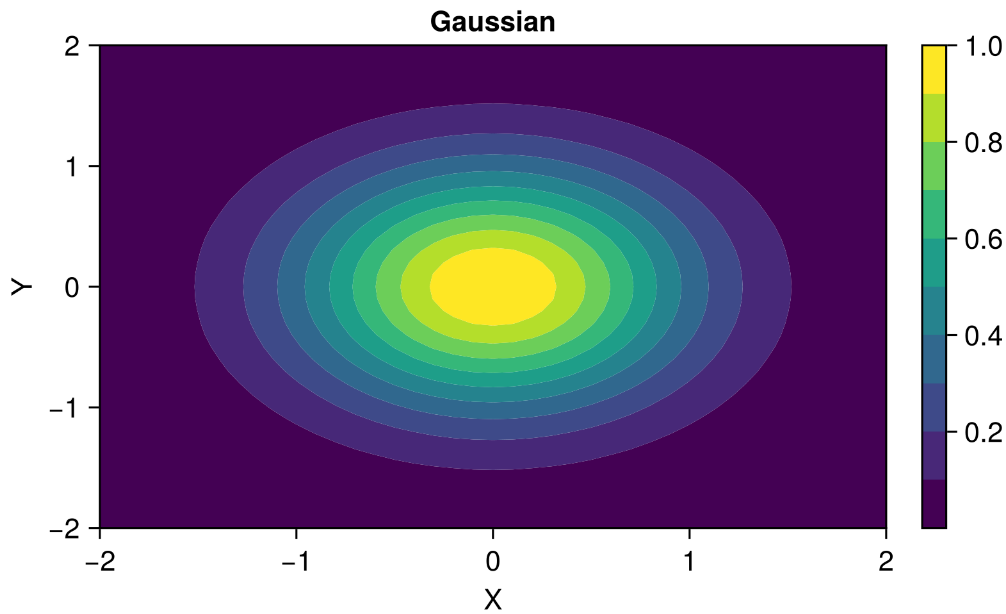
4.2 Subplots

```
fig = Figure(size=(700, 500))

ax1 = Axis(fig[1, 1], title="Sine")
lines!(ax1, x, sin.(x))

ax2 = Axis(fig[1, 2], title="Cosine")
lines!(ax2, x, cos.(x))

ax3 = Axis(fig[2, 1], title="Tangent", limits=(nothing, (-5, 5)))
lines!(ax3, x, tan.(x))

ax4 = Axis(fig[2, 2], title="Random")
scatter!(ax4, randn(20), randn(20))

fig
```



4.3 Contour Plots

```
x_cont = -2:0.1:2
y_cont = -2:0.1:2
z_cont = [exp(-(xi^2 + yi^2)) for xi in x_cont, yi in y_cont]

fig = Figure()
ax = Axis(fig[1, 1], xlabel="X", ylabel="Y", title="Gaussian")
co = contourf!(ax, x_cont, y_cont, z_cont, levels=10)
Colorbar(fig[1, 2], co)
fig
```



4.4 Geographic Maps

> **Under Construction**
>
> Geographic mapping examples with GeoMakie.jl will be added soon.

4.4.1 Coordinate Systems

```
# Example with projected coordinates
using CoordinateTransformations
using Proj4

# Transform from WGS84 to UTM
wgs84 = Proj4.Projection("+proj=longlat +datum=WGS84")
utm35 = Proj4.Projection("+proj=utm +zone=35 +datum=WGS84")
```

4.4.2 Plotting Geophysical Data

```julia
# Gravity anomaly map example
using CairoMakie

# Create synthetic gravity data
nx, ny = 50, 50
x_grav = range(0, 10, length=nx)
y_grav = range(0, 10, length=ny)
gz = [10 * exp(-((xi-5)^2 + (yi-5)^2)/2) for xi in x_grav, yi in y_grav]

fig = Figure()
ax = Axis(fig[1, 1],
          xlabel="Easting (km)",
          ylabel="Northing (km)",
          title="Bouguer Gravity Anomaly")
hm = heatmap!(ax, x_grav, y_grav, gz, colormap=:RdBu)
Colorbar(fig[1, 2], hm, label="mGal")
fig
```

5 Neural Networks

⚠ Under Construction

This chapter is currently being developed. Check back soon for updates!

5.1 Introduction to Neural Networks

Coming Soon

- Perceptrons and activation functions
- Feedforward networks
- Backpropagation

5.2 Flux.jl Basics

Coming Soon

- Installing Flux.jl
- Building your first neural network
- Training loops

5.3 Deep Learning Architectures

5.3.1 Convolutional Neural Networks (CNNs)

ℹ Coming Soon

- Convolution operations
- Pooling layers
- Image classification for geoscience

5.3.2 Recurrent Neural Networks (RNNs)

ℹ Coming Soon

- LSTM and GRU cells
- Time series prediction
- Sequence modeling for seismic data

5.3.3 Autoencoders

ℹ Coming Soon

- Encoder-decoder architecture
- Variational autoencoders
- Dimensionality reduction

5.4 Training Best Practices

ℹ Coming Soon

- Data preprocessing and normalization
- Batch training
- Learning rate scheduling
- Regularization techniques

5.5 GPU Computing with CUDA.jl

> ℹ Coming Soon
>
> - GPU setup
> - Moving data to GPU
> - Performance optimization

6 Inverse Modeling

> ⚠ Under Construction
>
> This chapter is currently being developed.

7 AI Surrogates

⚠ 🚧 Under Construction

This chapter is currently being developed.

7.0.1 DeepONet

ℹ Coming Soon

- Branch and trunk networks
- Operator learning framework
- Geophysical applications

7.1 Gaussian Processes

ℹ Coming Soon

- GP fundamentals
- Kernel selection
- Scalable GPs

7.2 Reduced-Order Models

ℹ Coming Soon

- Proper Orthogonal Decomposition (POD)
- Dynamic Mode Decomposition (DMD)
- Neural network-enhanced ROM

7.3 Uncertainty Quantification

ℹ Coming Soon

- Epistemic vs aleatoric uncertainty
- Ensemble methods
- Bayesian neural networks

7.4 Multi-Fidelity Modeling

ℹ Coming Soon

- Combining cheap and expensive models
- Transfer learning
- Active learning for surrogates

8 Applications

⚠ Under Construction

This chapter is currently being developed.